

RESTful Principles and Best Practices

For a **3-year Java/Spring Boot interview**, don't just memorize definitions. You should be able to explain **what REST is**, **why you use a particular HTTP method/status code**, and **how you design an API**.

1. What is REST?

REST = Representational State Transfer.

It is an architectural style for designing network APIs around **resources**.

For example, in a payment application:

```
Customer
Payment
Transaction
Refund
```

These become resources exposed through URLs.

```
GET    /payments/123
POST   /payments
PUT    /payments/123
DELETE /payments/123
```

The important idea is:

URL identifies the resource; HTTP method describes the operation.

2. REST Constraints

There are six fundamental REST constraints.

1. Client-Server

Frontend/client and backend/server should be separated.

```
React / Mobile App
  ↓
REST API
  ↓
Spring Boot
  ↓
Database
```

The client doesn't need to know how the server stores data.

2. Stateless

Each HTTP request should contain everything the server needs to process it.

The server should not depend on previous requests being stored in its session.

Example:

```
GET /payments/123
Authorization: Bearer <token>
```

The server validates the token for that request.

Interview question

What does stateless mean in REST?

The server does not maintain client session state between requests. Each request contains the information necessary for the server to process it.

This is particularly useful in distributed systems because any instance can handle the request.

3. Cacheable

Responses should indicate whether they can be cached.

For example:

```
Cache-Control: max-age=300
```

A payment status might need careful cache-control because stale information can be dangerous.

4. Uniform Interface

The API should follow consistent conventions.

For example:

```
GET    /payments/123
POST   /payments
PUT    /payments/123
DELETE /payments/123
```

Don't design one endpoint like:

```
POST /getPayment
POST /createPayment
POST /deletePayment
```

when standard HTTP methods already express these operations.

5. Layered System

The client doesn't necessarily know whether it is communicating directly with your service.

There may be:

```
Client
↓
API Gateway
```

```
↓
Load Balancer
↓
Payment Service
↓
Database
```

Each layer has a responsibility.

6. Code on Demand

This is an **optional** REST constraint and is less commonly used in modern APIs.

The server can send executable code to the client, such as JavaScript.

For a typical Spring Boot backend interview, know the concept but don't spend much preparation time on it.

3. Resource-Oriented URL Design

Use **nouns**, not verbs.

Good

```
GET /payments/123
POST /payments
GET /customers/100/payments
```

Avoid

```
GET /getPayment/123
POST /createPayment
POST /processPayment
```

The operation is represented by the HTTP method.

4. HTTP Methods

You should know these very well.

Method	Purpose	Example
GET	Retrieve resource	GET /payments/123
POST	Create/process	POST /payments
PUT	Replace/update resource	PUT /payments/123
PATCH	Partial update	PATCH /payments/123
DELETE	Delete resource	DELETE /payments/123

PUT vs PATCH

Suppose:

```
{  
  "amount": 1000,  
  "currency": "INR",  
  "status": "SUCCESS"  
}
```

PUT generally represents replacing the resource representation.

PATCH represents changing only specific fields.

```
PATCH /payments/123
```

```
{  
  "status": "FAILED"  
}
```

5. Idempotency

This is **extremely important for your payment project**.

An operation is idempotent if repeating the same request has the same intended effect as performing it once.

For example:

```
PUT /customers/123
```

Setting the same customer address repeatedly should result in the same final state.

But POST?

POST is generally **not inherently idempotent**.

For payment APIs, you may need to implement idempotency explicitly.

```
POST /payments  
Idempotency-Key: 7a81c...
```

First request:

```
Payment created → SUCCESS
```

Same request again:

```
Existing idempotency key found  
→ Return existing transaction  
→ Don't process payment again
```

Interview question

Why is idempotency important in payment systems?

Because network failures can cause a client to retry a request even when the original payment was

already processed. Without idempotency, the customer could potentially be charged twice.

6. HTTP Status Codes

You should know these without hesitation.

Success

```
200 OK
201 Created
202 Accepted
204 No Content
```

Client errors

```
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Content
```

Server errors

```
500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
504 Gateway Timeout
```

7. Important status-code distinctions

401 vs 403

401 → authentication is missing/invalid.

```
"I don't know who you are."
```

403 → authenticated but not permitted.

```
"I know who you are, but you cannot do this."
```

400 vs 422

400 Bad Request generally means the request is malformed or invalid.

422 is useful when the request is syntactically valid but fails semantic/business validation.

Example:

```
{
  "amount": -500
}
```

The JSON is valid, but the business value is invalid.

The exact choice should follow your organization's API standards; consistency matters more than blindly choosing one.

409 Conflict

Useful when the request conflicts with the current resource state.

Example:

```
Payment already processed
```

could reasonably return:

```
409 Conflict
```

depending on your API contract.

8. Request and Response Design

Don't expose your database entity directly.

Entity

```
@Entity
class Payment {
    private Long id;
    private BigDecimal amount;
    private String customerId;
}
```

Create DTOs:

```
class PaymentRequest {
    private BigDecimal amount;
    private String customerId;
}
```

```
class PaymentResponse {
    private String transactionId;
    private String status;
}
```

Why?

- API contract is separated from DB structure.
 - You avoid exposing internal fields.
 - API can evolve independently.
 - Validation becomes cleaner.
-

9. Validation

Validate input at the API boundary.

```
@PostMapping("/payments")
public ResponseEntity<PaymentResponse> createPayment(
    @Valid @RequestBody PaymentRequest request) {
    ...
}
```

Example:

```
@NotNull
@Positive
private BigDecimal amount;
```

For an amount:

```
null      → invalid
0         → invalid
-100      → invalid
100.00    → valid
```

Business validation should generally remain in the service/domain layer rather than putting all rules into the controller.

10. Consistent Error Response

Don't return random error structures from different APIs.

Bad:

```
{
  "error": "failed"
}
```

Better:

```
{
  "code": "PAYMENT_NOT_FOUND",
  "message": "Payment does not exist",
  "timestamp": "2026-08-30T06:00:00Z",
  "traceId": "abc-123"
}
```

Your project may use a different structure, but it should be consistent.

Spring Boot commonly handles this through:

```
@RestControllerAdvice
```

and:

```
@ExceptionHandler
```

11. Pagination

Don't return thousands/millions of records in one response.

Instead:

```
GET /payments?page=0&size=20
```

or cursor-based pagination for suitable high-volume systems.

Response:

```
{
  "content": [
    ...
  ],
  "page": 0,
  "size": 20,
  "totalElements": 1500
}
```

For high-scale systems, **cursor/keyset pagination** can be preferable to large offset-based queries.

12. Filtering, Sorting and Searching

Use query parameters.

```
GET /payments?status=SUCCESS
```

Multiple filters:

```
GET /payments?status=SUCCESS&currency=INR
```

Sorting:

```
GET /payments?sort=createdAt,desc
```

Pagination:

```
GET /payments?page=0&size=20
```

Avoid creating a separate endpoint for every combination.

13. API Versioning

When you make breaking changes, you need a versioning strategy.

One common approach:

```
/api/v1/payments
/api/v2/payments
```

Another is header-based versioning.

The important thing in an interview is to explain **why versioning is needed**:

- Existing consumers should continue working when the API contract changes.

14. Authentication and Authorization

Authentication:

- Who are you?

Authorization:

- Are you allowed to perform this operation?

Example:

```
POST /payments
↓
JWT validation
↓
Authenticated
↓
Role/permission check
↓
Payment processing
```

Don't put sensitive information such as passwords or secrets into query parameters.

Use:

```
Authorization: Bearer <token>
```

over HTTPS.

15. HTTPS

REST APIs carrying sensitive/payment information should use HTTPS/TLS.

Never send sensitive payment information over plain HTTP.

Also avoid putting secrets or sensitive payment data into logs.

16. Timeout, Retry and Downstream APIs

Suppose your BPS service calls another payment system:

```
BPS
```

↓
Payment Gateway

The gateway doesn't respond.

Your API should have:

- connection timeout
- read timeout
- appropriate retry policy
- circuit breaker where appropriate
- meaningful error handling

But **don't blindly retry payment operations**.

For example:

```
Request → Gateway processed payment
        → Response lost
        → BPS sees timeout
        → Retry
```

That can potentially cause duplicate processing.

Therefore combine **idempotency + safe retry strategy**.

17. REST API Security Best Practices

For your payment project, mention:

- HTTPS/TLS
- authentication
- authorization
- input validation
- SQL injection prevention
- rate limiting
- secure headers
- sensitive-data masking
- secret management
- audit logging
- appropriate CORS configuration
- avoid exposing internal implementation details

18. REST vs SOAP

This may come up because banking/payment projects sometimes use both.

REST

```
Usually HTTP + JSON
Lightweight
Resource-oriented
Common for modern APIs
```

SOAP

XML-based
Formal contract via WSDL
Built-in enterprise standards around messaging/security
Common in legacy/enterprise integrations

Don't say:

“REST is always better.”

The correct answer is that the choice depends on **integration requirements, contracts, ecosystem and system constraints**.

19. REST Best-Practice Architecture in Spring Boot

A typical implementation:

```
HTTP Request
  ↓
Controller
  ↓
DTO + Validation
  ↓
Service
  ↓
Business Logic
  ↓
Repository / External Client
  ↓
Database / Downstream System
  ↓
Response DTO
  ↓
HTTP Response
```

Example:

```
@RestController
@RequestMapping("/api/v1/payments")
public class PaymentController {

    @PostMapping
    public ResponseEntity<PaymentResponse> createPayment(
        @Valid @RequestBody PaymentRequest request) {

        PaymentResponse response = paymentService.create(request);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(response);
    }
}
```

20. What the interviewer can ask you

For your BPS project, I would especially prepare these:

Basic

Q: What is REST?

REST is an architectural style for building distributed systems around resources using a uniform interface, commonly HTTP methods and representations such as JSON.

Q: What is statelessness?

Each request contains the information needed to process it, and the server does not rely on stored client session state between requests.

Q: GET vs POST?

GET retrieves a resource; POST generally creates a resource or triggers an operation that isn't naturally represented as an idempotent resource replacement.

Q: PUT vs PATCH?

PUT generally replaces the resource representation, whereas PATCH applies a partial modification.

Intermediate

Q: Why shouldn't you use verbs in REST URLs?

Because HTTP methods already represent the operation.

```
Good:
POST /payments

Avoid:
POST /createPayment
```

Q: What is idempotency?

Repeating the same operation produces the same intended final effect. It is especially important for payment retries.

Q: Why use DTO instead of entity?

To separate the external API contract from the persistence model and avoid exposing internal fields.

Project-level

Q: Explain one REST API from your BPS project.

You should answer:

```
Endpoint
↓
HTTP method
↓
Request DTO
↓
```

```
Validation
↓
Authentication/Authorization
↓
Controller
↓
Service
↓
Business validation
↓
DB/Downstream API
↓
Transaction handling
↓
Response
↓
Exception handling
↓
Logging/trace ID
```

Then be able to explain **why each step exists**.

What I would consider “interview ready” for 3 years

You should be able to design this without hesitation:

```
POST /api/v1/payments
GET  /api/v1/payments/{paymentId}
GET  /api/v1/payments?status=SUCCESS&page=0&size=20
PATCH /api/v1/payments/{paymentId}
```

And explain:

REST constraints → resource naming → HTTP methods → status codes → DTOs → validation → authentication → idempotency → transactions → error handling → pagination → versioning → timeout/retry → logging.

For your **BPS payment project**, the highest-value topics are **REST + idempotency + transaction management + downstream failures + security** rather than just memorizing HTTP methods.